Final Project Proposal: Infinite Hexagonal Tic-Tac-Toe Engine

Jeremiah Young
Department of Electrical and Computer Engineering
Oklahoma State University
Stillwater, Oklahoma
jeremiah.young@okstate.edu

***Abstract*—This paper was typeset using LaTeX and acts as a project proposal for a hexagon tic-tac-toe solver. To address the computational challenges of an infinite board and an unbounded state space, the solver will utilize Monte Carlo Tree Search and a heuristic pruning method that utilizes an "influence field" to evaluate an Upper Confidence Bound for Trees.**

I. Project Description

This project consists of making a chess-like engine for an unsolved 6 in a row infinite tiling hexagon tic-tac-toe, providing move suggestions, forced wins to some depth, and an evaluation score for the current position. I chose this topic to investigate how numerical methods can resolve unbounded state spaces where deterministic solvers are too computationally complex. To do this, **Monte Carlo Tree Search** will be used to simulate random board states. To provide a form of heuristic pruning, a field of influence will be calculated with a **vectorized field equation** ($E = \sum \frac{c}{d^2}$). To avoid this calculation taking infinite time, the engine utilizes a **piecewise evaluation strategy** where the board is chunked into local coordinate meshes that are dynamically calculated. To round this project out, parallel processing will be implemented with carefully chosen random seeds to ensure the convergence of evaluation scores is accelerated.

II. Project Goals

The primary objective of this project is to create a highly efficient analysis engine for infinite Hexagonal Tic-Tac-Toe. The engine will demonstrate the following capabilities:

- **Strategic Move Suggestion:** If the user requests a move recommendation, the program shall output the most statistically favorable coordinate.
- **Dynamic Position Evaluation:** If the board state changes, the program shall display a real-time numerical advantage score based on the influence field.
- **Forced Win Detection:** If a winning sequence exists within a reachable depth, the program shall notify the user of a deterministic "Mate in N" state.
- **Asynchronous Computation:** If the engine is performing heavy simulations, the program shall maintain a responsive UI, allowing the user to interact with the board without input lag.

III. Requirements

The success of the project will be measured against the following testable specifications:

1) **Vectorized Field Efficiency:** The vectorized field calculation must perform at least $20\times$ faster than a standard Python `for`-loop when evaluated over a 50×50 grid.
2) **Search Complexity Scaling:** Win-state detection must demonstrate $O(1)$ time complexity relative to the total number of pieces on the board.
3) **Statistical Convergence:** The engine must show that running N parallel simulations reduces the standard error of the evaluation score by a factor approaching $1/\sqrt{N}$.
4) **Algorithmic Accuracy:** When presented with a known geometric trap, the engine must identify the winning move in 100% of test cases in a time approaching 2 seconds.
5) **Heuristic Pruning Efficiency:** The influence-based pruning mechanism must successfully exclude at least 90% of legal moves in a 50×50 evaluation chunk while maintaining a 100% retention rate for moves that contribute to a detected forced-win sequence.

References

[1] webgoatguy, "can tic-tac-toe, with hexagons?," *YouTube*, Mar. 5, 2026. [Online]. Available: http://www.youtube.com/watch?v=Ob6QINTMIOA
[2] webgoatguy, "hexagonal tic-tac-toe is ridiculous," *YouTube*, Mar. 18, 2026. [Online]. Available: http://www.youtube.com/watch?v=EzLzkcRZurk